

BLOC: Byzantine-resilient multi-proposer block construction in Ethereum

PIC2 - Master in Computer Science and Engineering
Instituto Superior Técnico, Universidade de Lisboa

Vasco Magalhães e Silva — ist199132*
vasco.magalhaes.silva@tecnico.ulisboa.pt

Advisor(s): Rodrigo Rodrigues, Christof Torres

Abstract Maximal Extractable Value (MEV) and transaction censorship remain fundamental challenges in Ethereum, arising from the visibility of pending transactions and the unilateral control that block proposers and builders exert over transaction inclusion and ordering. Existing approaches face fundamental trade-offs between communication efficiency, privacy guarantees, and practical deployability without a hard-fork to Ethereum’s current protocol. As a result, users continue to face execution manipulation, exclusion risks, and growing centralization pressures in block production.

We propose an encrypted multi-proposer block construction mechanism that combines recent advances in batched threshold encryption with Byzantine-resilient multi-proposer coordination implemented via Distributed Validator Technology (DVT), where a single logical validator is operated by a committee of independent operators using threshold cryptography. Transactions are encrypted prior to mempool admission and remain opaque throughout ordering, while the DVT operators jointly determine block contents using a Byzantine Fault Tolerant (BFT) consensus-based state machine replication protocol. Only after ordering is determined does the committee collaboratively threshold decrypt the selected batch, ensuring that inclusion and ordering decisions are content-agnostic and that no single proposer can exert unilateral control.

The primary objective of this work is to design, implement, and evaluate the practicality of this construction within Ethereum’s existing infrastructure. By grounding encrypted multi-proposer block construction in deployable mechanisms, this work aims to advance a concrete and practical solution to mitigate censorship and MEV on Ethereum.

Keywords — Blockchain, MEV, Ethereum, Batched Threshold Encryption, Distributed Validator Technology

*I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa (<https://nape.tecnico.ulisboa.pt/en/apoio-ao-estudante/documentos-importantes/regulamentos-da-universidade-de-lisboa/>).

Contents

1	Introduction	3
1.1	Work Objectives	4
1.2	Expected Contributions	4
2	Related Work	5
2.1	MEV Auction Platforms	5
2.2	Shutter: Epoch-Based Threshold Encryption and the Ethereum L1 Roadmap	6
2.3	FairBlock: bandwidth-efficient per-block threshold decryption	7
2.4	F3B: per-transaction threshold encryption architecture	8
2.5	BlindPerm: encrypted mempool with permutation	9
2.6	Obol / Nethermind: using DV clusters for encrypted transaction mempools	10
3	Solution: DVT-Based Encrypted Mempool Block Construction	11
3.1	Encrypted Transactions and DV Cluster Block Construction	11
3.2	Challenges and Practical Considerations	14
4	Evaluation	15
4.1	Evaluation Setup and Prototype	15
4.2	RQ1: Is the protocol practical within Ethereum’s slot-level timing constraints?	15
4.3	RQ2: What overhead does multi-proposer coordination and cryptography introduce?	16
4.4	RQ3: How robust is the protocol under faults and adversarial behavior?	17
4.5	RQ4: What is the economic cost of the protocol for users and validators?	17
5	Work Schedule	18
6	Conclusion	18
	Bibliography	20

1 Introduction

Public blockchains such as Ethereum provide strong security guarantees through permissionless consensus with globally replicated state and native execution of general-purpose smart contracts. These systems have enabled a rapidly growing decentralized finance (DeFi) ecosystem encompassing decentralized exchanges, lending markets, stablecoins, and tokenized real-world assets. As of early 2026, DeFi secures over \$120 billion in total value locked [1], underscoring the importance of transaction integrity and fairness.

A fundamental limitation of today’s public blockchains is the lack of native control over transaction inclusion and ordering prior to execution. Transactions are broadcast in plaintext to a public mempool, where they are visible to block proposers, builders, and specialized trading bots. This transparency enables maximal extractable value (MEV), first systematically identified by Daian et al. [2], who showed that transaction ordering freedom gives rise to adversarial behaviors such as front-running, sandwich attacks, back-running, and targeted censorship. In a sandwich attack, an adversary observes a pending user transaction (e.g., a large trade on a decentralized exchange) and inserts two transactions of its own—one immediately before and one immediately after the victim’s transaction—artificially moving the price to extract profit from the price impact of the victim’s transaction. For example, from September 2022 to June 2024, predatory sandwich attackers made a profit of more than \$41 million [3]. These behaviors impose direct financial losses on users, inflate transaction fees, and introduce systemic inefficiencies by incentivizing competition over ordering rather than execution correctness.

In practice, the dominant response to MEV has been the emergence of MEV auction platforms, most notably Proposer–Builder Separation (PBS) and its off-chain instantiation via MEV-Boost, middleware developed to allow proposers to access blocks from a marketplace of builders [4]. PBS restructures block production by separating transaction ordering from block proposal: specialized builders construct candidate blocks and compete in auctions, while proposers select blocks based on bids without direct visibility into transaction contents. This design improves capital efficiency and reduces some proposer-side incentives, but leaves the underlying sources of MEV intact and shifts ordering power and transaction visibility to builders and relays.

Empirical evidence indicates that this shift has resulted in significant centralization, with a small number of builders dominating block production and exerting disproportionate influence over transaction ordering [5]. Moreover, current PBS deployments rely on trusted third-party relays, introducing additional attack surfaces. In April 2023, a malicious proposer exploited a vulnerability in a relay implementation, resulting in the theft of approximately \$25 million from multiple MEV searchers, highlighting the systemic risks introduced by these trusted components [6]. We defer a detailed discussion of PBS, MEV auction platforms, and related approaches to Section 2.

Motivated by these shortcomings, a second line of work seeks to limit proposer and builder discretion by enforcing content-agnostic ordering, most prominently through encrypted mempools and threshold decryption [7–13]. By encrypting transactions prior to mempool admission and revealing their contents only after ordering has been fixed, these approaches aim to prevent ordering decisions from depending on transaction semantics. However, existing encrypted-mempool designs face substantial challenges. Basic threshold encryption incurs communication costs proportional to both the number of transactions and the size of the decryption committee $\mathcal{O}(nB)$, where n denotes the committee size and B the number of transactions per block, making it impractical under tight block production deadlines as outlined in [13]. Epoch or block-targeted optimizations reduce communication overhead but introduce transaction exposure if encrypted transactions are not included within their intended window, violating pending-transaction privacy as pointed out in [13]. Other proposals require execution delays until finality, introduce new transaction types, and ultimately, their deployability would require core changes to the Ethereum protocol. We survey these approaches and their limitations in detail in Section 2.

Recent work on batched threshold encryption significantly improves the efficiency of encrypted mempools by allowing a committee to decrypt an entire block of transactions with communication independent of block size, while preserving the privacy of non-included transactions [12, 13]. While these primitives resolve a key bottleneck, they don’t cover transaction inclusion and censorship resistance since metadata remains in plaintext nor do they outline concrete deployment within the current Ethereum ecosystem.

Nevertheless, we will rely on the cryptographic primitives presented in these solutions for our implementation.

This proposal argues that meaningful MEV mitigation can be addressed through transaction confidentiality, making transaction payloads opaque and hard to exploit, and proposer centralization, distributing block building to avoid single-party discretion regarding transaction inclusion which can lead to censorship. We therefore propose an encrypted multi-proposer block construction mechanism that combines batched threshold encryption with Byzantine-resilient multi-proposer coordination. Transactions are encrypted upon submission and remain opaque throughout the mempool and ordering phases. A committee of proposers jointly determines transaction inclusion and ordering without access to transaction contents, and only after a block is finalized does the committee collaboratively release decryption shares for the selected batch. By construction, ordering decisions are determined before transaction contents are revealed, and no single party can unilaterally censor or manipulate the block.

A central design goal of this work is deployability. Rather than requiring intrusive changes to Ethereum’s consensus or execution layers, we leverage Distributed Validator Technology (DVT), which is employed by organizations such as ssv.network and obol.org, to implement the multi-proposer protocol within a single logical validator. DVT splits a validator’s signing key across multiple independent operators using threshold cryptography, allowing them to collectively perform proposer duties while remaining fully compatible with the existing beacon-chain interface. This enables incremental deployment of encrypted multi-proposer block construction without hard forks, distinguishing our approach from prior proposals.

1.1 Work Objectives

The primary objective of this work is to design, analyze, and evaluate an encrypted multi-proposer block construction mechanism that mitigates MEV by jointly addressing transaction confidentiality and proposer centralization, while remaining compatible with Ethereum’s existing infrastructure. This work aims to integrate encrypted mempools and a multi-proposer design into a single coherent protocol in which transaction ordering is fixed before transaction contents are revealed and no single party exercises unilateral control over block construction.

Concretely, this work pursues the following objectives. First, we aim to construct a block-building pipeline in which transactions remain encrypted throughout the mempool and ordering phases and are decrypted only after their relative order has been irrevocably determined. This objective is achieved by leveraging batched threshold encryption to ensure scalability and to preserve pending-transaction privacy. Second, we seek to replace the single designated proposer with a Byzantine-resilient committee of proposers that jointly determine transaction inclusion and ordering without access to transaction contents, thereby eliminating the proposer’s last-mover advantage. Third, we aim to demonstrate that encrypted multi-proposer block construction can be deployed incrementally, without consensus-layer changes, by embedding the protocol within a DVT cluster and preserving full backward compatibility with Ethereum.

1.2 Expected Contributions

The expected contributions of this work are the following. First, we survey and organize the state of the art in encrypted-mempool and MEV-mitigation protocols, situating our design within the broader landscape of proposals. This analysis clarifies how our proposal relates to prior work and highlights the specific trade-offs addressed by our design choices.

Second, we present a block construction design that composes batched threshold encryption with multi-proposer coordination, addressing both content-based MEV and proposer monopoly within a unified framework. This contribution clarifies the complementary roles of cryptographic confidentiality and distributed ordering authority in MEV mitigation.

Third, our objective is to provide a protocol specification and security analysis tailored to the MEV setting, capturing adversarial behaviors such as ordering manipulation, selective decryption, and censorship attempts by subsets of proposers or validators.

Finally, we intend to evaluate the practicality of encrypted multi-proposer block construction, focusing on latency, computational overhead, and feasibility within Ethereum’s slot timing constraints.

2 Related Work

When it comes to MEV countermeasures we can break down the state of the art into two schools of thought as per [3]: (i) make MEV extraction efficient, decentralized, and transparent ensuring all block proposers have the same capability of extracting MEV (ii) Limiting proposer control over the order of transactions. My proposal builds on the second line of work, specifically using a threshold decryption approach, but focuses specifically on how Distributed Validator Technology (DVT) can be repurposed as a threshold decryption committee that remains compatible with today’s Ethereum beacon-chain interface, avoiding a hard fork while still using the public mempool.

(i)

2.1 MEV Auction Platforms

Before turning to threshold-encryption-based countermeasures, it is essential to describe the dominant class of practical MEV mitigation infrastructure deployed today: MEV auction platforms. These systems correspond to point (i) in the introduction—approaches that aim to make MEV extraction efficient, transparent, and competitively accessible, rather than attempting to eliminate MEV outright.

The existence of MEV as a competitive market was first systematically identified by Daian et al. [2]. The authors show that transaction ordering on Ethereum naturally gives rise to priority gas auctions (PGAs), in which searchers compete for ordering priority by bidding up transaction fees. In this model, MEV extraction emerges endogenously as a fee-driven competition for blockspace, leading to inefficiencies such as bidding wars, redundant computation, and consensus instability.

Building on this diagnosis, Proposer–Builder Separation (PBS) [14] was later proposed as a protocol-level restructuring of block production. PBS makes the implicit PGA-based competition explicit by separating the roles of transaction ordering and block proposal. Specialized builders compete to construct the most profitable block, while proposers select among these blocks based solely on bids, without direct access to transaction contents. This design aims to democratize access to MEV opportunities by allowing any participant to compete in block construction, while limiting the unilateral power of the proposer.

MEV-Boost and the PBS Supply Chain

After the Merge, Ethereum participants adopted MEV-Boost [4] as an off-chain implementation of proposer–builder separation (PBS). MEV-Boost instantiates PBS through a builder–relay–proposer pipeline involving the following participants:

- **Searchers**, who identify MEV opportunities and submit ordered transaction bundles to builders.
- **Builders**, who aggregate public mempool transactions and privately submitted bundles into candidate blocks and compute bids reflecting the block’s expected extractable value.
- **Relays**, which mediate sealed-bid auctions by accepting blocks from builders, verifying their validity, and forwarding only block headers and bids to proposers.
- **Proposers**, who select the highest-bidding block without direct visibility into its contents and publish it on-chain.

While MEV-Boost decentralizes the builder role relative to proposers, it does not decentralize the builder market itself. In fact, empirical evidence shows that two builders routinely win over 85–90% of slots, creating one of the most centralized components of Ethereum’s infrastructure [5].

Additional trust assumptions persist:

- users must trust relay-builder pairs to not censor, reorder, or leak transactions;
- builders retain full visibility into transaction contents and can reorder within the block;
- proposers receive no cryptographic guarantee of inclusion fairness.

Thus MEV-Boost aligns with the first school of MEV countermeasures: it optimizes and organizes MEV extraction, but does not reduce proposer discretion or provide content-agnostic ordering guarantees.

Private RPCs and User-Facing MEV Mitigation

A parallel class of practical MEV mitigation mechanisms operates at the RPC layer through so-called MEV protection RPCs, such as Flashbots Protect and MEV-Blocker [15]. These systems are designed to protect users from adversarial MEV strategies—most notably frontrunning and sandwich attacks—by allowing transactions to be submitted privately, bypassing the public mempool where pending transactions are globally visible.

Rather than broadcasting transactions to all network participants, private RPCs act as order flow aggregators, collecting user-submitted transactions and selectively routing them through order flow auctions (OFAs). In an OFA, access to a user’s transaction is auctioned among competing searchers, who submit backrunning strategies and associated bids reflecting the value they can extract. The RPC selects the most profitable bid and forwards the resulting bundle to block builders for inclusion. In this context, order flow refers to the transaction data and associated execution information controlled by the RPC prior to block inclusion, which can be leveraged to construct MEV strategies. To limit adversarial behavior, these systems typically restrict searcher interaction with order flow to backrunning only, reducing value extraction to non-harmful post-execution arbitrage and liquidation strategies. Ultimately, the winning bid is split between the user and the builder, with the user typically receiving a majority share in the form of a rebate.

Empirical measurements show that different RPCs employ materially different OFA designs, fee models, and routing strategies, leading to heterogeneous outcomes in inclusion latency, execution quality, and user rebates [15].

While effective at mitigating public-mempool MEV and improving user experience in practice, private RPCs introduce new trust assumptions and centralization pressures. The RPC operator gains full visibility into user transactions and exercises discretionary control over which searchers and builders may participate, how much information is revealed, and which bundles are ultimately forwarded to builders. As a result, private RPCs do not provide cryptographic guarantees of ordering neutrality, inclusion fairness, or censorship resistance. Instead, they shift trust from proposers and builders to a small set of centralized order flow intermediaries.

Limitations and Motivation for Threshold Encryption

The MEV auction platforms presented above dominate current practice and have ultimately led to the centralization of trust. They do not prevent builders or relays from censoring or excluding transactions, nor do they provide a decentralized and cryptographically sound approach to combat the prevalence of MEV in the Ethereum ecosystem.

This motivates the second school of countermeasures, content-agnostic ordering and threshold-encrypted mempools, which seek to remove proposer and builder discretion over transaction ordering.

The remainder of this section surveys these approaches.

(ii)

2.2 Shutter: Epoch-Based Threshold Encryption and the Ethereum L1 Roadmap

Shutter [10, 16] proposes an encrypted-mempool architecture based on threshold identity-based encryption (IBE), in which transaction confidentiality is enforced by encrypting transactions to a publicly known epoch identifier.

In the Shutter design, trust is distributed across the keyper committee rather than vested in a single authority. During a one-time setup phase, keypers jointly run a distributed key generation (DKG) protocol to produce a master public key mpk , which is published on-chain, and a corresponding master secret key msk , which is never reconstructed and exists only in shared form among the committee. In principle, the keyper committee may be composed of any set of independent operators—such as validators, infrastructure providers, or other decentralized actors—provided that a threshold of them behaves honestly.

Transactions are encrypted using identity-based encryption (IBE) with respect to a known time interval, called an eon. For an eon with index $i \in \mathbb{N}$, the eon index itself serves as the IBE identity, and the corresponding eon private key is defined as $sk_Q := msk \times H_1(i)$, where H_1 maps identities to the appropriate group. Users encrypt transaction payloads as $\text{Enc}_{ID}(mpk, H_1(i), T)$, binding ciphertexts to a specific eon.

Each keyper can locally derive a share of the eon private key from its share of the master secret. After the eon ends, keypers broadcast their shares, and once a threshold is reached, the eon private key sk_Q is reconstructed, enabling batch decryption of all transactions submitted during that eon. In the reference implementation, eons span 10 blocks (approximately 2.5 minutes).

In practice, the live Shutter deployment instantiates the keyper committee as a fixed set of external operators running dedicated keyper nodes outside of Ethereum’s validator set, forming a separate trust domain responsible solely for threshold decryption.

While this design provides strong ordering privacy within an epoch, it introduces some limitations in the context of Ethereum L1. Encryption and decryption are tied to coarse-grained eons (10 blocks) rather than individual blocks or slots, presenting a trade-off which might not be suitable for many users. This epoch-based decryption also implies that all transactions encrypted for an eon become decryptable once the eon key is released, including transactions that were not ultimately included, leading to non-inclusion leakage, a common issue in IBE based proposals which ultimately leaves these transactions vulnerable to MEV. This issue has been addressed in subsequent work [12, 13] providing batched threshold encryption as a possible alternative where non-included transactions are not revealed, an approach that is used in our proposal.

The Shutter “Road Towards an Encrypted Mempool” white paper takes the Shutter core protocol and outlines a concrete roadmap for integrating a threshold encrypted mempool into Ethereum’s PBS transaction supply chain and eventually an in-protocol integration. The roadmap is structured in steps:

1. RPC-level integration: introduce an `eth_sendEncryptedTransaction` endpoint (e.g., via MEV Blocker), allowing users to submit encrypted transactions that are decrypted off-chain by Shutter keypers and then forwarded to builders.
2. PBS integration via proposer commitments: builders and relays operate on encrypted payloads, while proposers commit to inclusion of ciphertexts before keys are released, ensuring inclusion fairness.
3. In-protocol implementation: long-term goal of integrating threshold encryption directly into Ethereum’s consensus layer.

The authors acknowledge several open challenges in this roadmap:

- Complex integration with PBS. The current fragmented builder–relay–proposer supply chain makes enforcing encrypted-transaction inclusion non-trivial; proposer commitments backed by slashing based economic security are proposed but not yet standardized in the ecosystem.
- Eventual need for protocol changes. The last point explicitly assumes in-protocol changes to Ethereum, requiring a hardfork, to fully realize an encrypted mempool.

2.3 FairBlock: bandwidth-efficient per-block threshold decryption

Similarly to Shutter, FairBlock [9] addresses MEV mitigation by encrypting user transactions under an identity-based encryption (IBE) scheme and having a committee, which may comprise users, validators,

decentralized oracle networks or DAOs, derive a block-specific decryption key. This key is then used to decrypt all transactions assigned to that block once their ordering has been finalized. FairBlock supports two deployment paths: a smart-contract-based design, where decryption and execution occur through a series of contract calls, and a consensus-layer integration, where validators natively participate in the protocol.

The central innovation is that, unlike conventional threshold-decryption systems where each committee member must produce a separate partial decryption for every transaction in the block, FairBlock leverages the linearity of IBE private keys so that each keeper produces only one share of the block’s private key, regardless of how many transactions the block contains. After these shares are aggregated, the resulting block key can decrypt all ciphertexts for that block without further communication. This transforms the communication cost from proportional to the number of transactions ($N \times S$) to proportional only to the size of the committee (S), which explains the substantial reduction in bandwidth. In their experiments, this design reduces bandwidth overhead by up to two orders of magnitude compared to naïve threshold decryption.

Similarly to Shutter, FairBlock’s main limitations are:

- **Transaction exposure if not included.** If a transaction fails to be included in the client-chosen block, it is undesirably revealed.
- **Metadata leakage.** Even with encrypted data fields, transaction metadata such as the sender identity can leak information, enabling speculative attacks based on trading patterns. This limitation is also present in our solution but is considered an acceptable trade-off with possible use of pseudonym sender addresses to mitigate the issue.

2.4 F3B: per-transaction threshold encryption architecture

F3B [8] proposes a blockchain architecture in which each transaction is encrypted under a fresh, per-transaction key and posted on-chain. A separate secret-management committee (SMC) is responsible for managing decryption: committee members observe encrypted transactions and release decryption shares only after the underlying consensus protocol has finalized the transaction’s ordering. The SMC may be composed of dedicated trustees, consensus validators, or a subset of validators selected per epoch, and is logically decoupled from the consensus layer. Once a threshold of shares is revealed, the transaction can be decrypted and executed. This per-transaction design avoids the chosen-block leakage present in earlier schemes such as FairBlock and Shutter, where users must target a future block or epoch and transactions may be revealed prematurely if inclusion is delayed.

To enforce correct behavior, F3B introduces an explicit staking and slashing mechanism for the secret-management committee (SMC). Committee members are required to lock collateral that can be slashed if a valid decryption share is revealed before finality. Premature disclosure is cryptographically provable because each decryption share is accompanied by a publicly verifiable proof of correctness that binds the share to a specific on-chain ciphertext; revealing such a share prior to finality produces undeniable, on-chain evidence of misbehavior that can only be generated by the corresponding committee member. As a result, third parties are incentivized to report misbehavior, making collusion among committee members economically unstable.

In the context of Ethereum L1, F3B still exhibits several important limitations:

- Execution is intentionally delayed until finality. While this prevents reorg-based ordering attacks, it introduces substantial practical latency, since Ethereum finality occurs after $64 \times 12 = 768$ seconds. The reported 0.03% overhead is measured relative to this finality window and may therefore overstate the practical efficiency of the design when compared to approaches that execute transactions at block inclusion, as the communication overhead introduced by per-transaction encryption keys is amortized by deferring decryption until finality and is acceptable only in this setting.
- The approach requires execution-layer modifications and the introduction of a new transaction type supporting encrypted, deferred execution.

2.5 BlindPerm: encrypted mempool with permutation

BlindPerm [7] proposes a framework for mitigating ordering-based MEV by combining an encrypted mempool with a post-commitment random permutation of transactions. Rather than fixing a single cryptographic construction, BlindPerm is deliberately agnostic to the concrete threshold-encryption primitive and instead specifies a set of functional requirements that encrypted mempool designs must satisfy. The framework distinguishes between (i) how transaction confidentiality is achieved prior to commitment and (ii) how a shared permutation is derived after commitment to reduce the profitability of adversarial ordering.

Threshold-encrypted mempool (baseline construction). In the baseline construction, BlindPerm assumes that the validator set jointly provides threshold security over the mempool. Validators first execute a distributed key generation (DKG) protocol to obtain a long-lived threshold public key, while each validator holds a share of the corresponding secret key, which is never reconstructed in full. Users encrypt their transaction payloads directly under this public key and submit the resulting ciphertexts to the mempool.

Once a block of encrypted transactions is proposed and committed by a BFT-style consensus protocol, each validator locally computes a ciphertext-specific partial decryption using its secret key share. These partial decryptions are broadcast, and any quorum of valid shares suffices to recover the plaintext transaction. Importantly, validators only produce partial decryptions for transactions that are actually included in a committed block, enforcing inclusion-conditional decryption and preventing premature disclosure of excluded transactions. In this design, the validators responsible for threshold decryption coincide with those responsible for block proposal and finality.

Alternative key-distribution mechanisms. Beyond this baseline approach, BlindPerm considers several alternative encrypted mempool instantiations that differ primarily in how transaction decryption keys are generated and distributed:

- **Secret sharing with post-verification**, where users encrypt transaction payloads symmetrically and distribute the corresponding encryption keys to validators using verifiable secret sharing. Validators defer key reconstruction until after block commitment, at which point any quorum can recover the symmetric key and decrypt the transaction.
- **Publicly verifiable secret sharing (PVSS)**, which strengthens the previous approach by enabling public verification of correct share generation and decryption via non-interactive zero-knowledge proofs. This improves robustness against faulty dealers or validators, at the cost of increased user-side computation and bandwidth.

Across all variants, the common design principle is that transaction contents remain hidden until after the block has been committed.

Post-commitment permutation. Independently of the chosen encryption primitive, BlindPerm introduces a post-commitment random permutation of transactions prior to execution. After a block is committed, validators derive a shared permutation seed, either via threshold BLS signatures over consensus-agreed data or via aggregation of reconstructed per-transaction secrets. Each validator locally applies a pseudorandom permutation to the committed transaction list using this seed, ensuring that execution order is decoupled from proposal order.

This permutation step significantly reduces the expected profitability of arbitrage and sandwich attacks by making the relative position of transactions unpredictable at proposal time, as supported by simulations on historical Ethereum data. It should be pointed out that these results might be biased by the naive assumption that MEV extraction approaches would have been the same if this system was in place.

Selective encryption and permutation. BlindPerm further proposes a selective encryption optimization in which only MEV-sensitive transactions are encrypted, while non-sensitive transactions are submitted in plaintext. Although plaintext transactions are visible prior to commitment, protection is still provided for encrypted transactions through the post-commitment permutation. Any adversary attempting to exploit an encrypted victim transaction must ensure its inclusion in the committed block; doing so necessarily adds randomness from the encrypted transaction into the permutation seed, preserving unpredictability of the execution order. As a result, even selective encryption suffices to protect MEV-sensitive transactions while reducing overall cryptographic overhead.

Limitations. Important limitations remain. Several BlindPerm instantiations rely on per-transaction threshold decryption, requiring validators to produce and disseminate partial decryption shares for each encrypted transaction in the block. This design can incur substantial communication and computation overhead at high transaction throughput, particularly when compared to schemes that reduce decryption costs by batching these operations across transactions as shown in previous protocols. In addition, residual attack surfaces persist: block producers may bypass the mechanism by including only a single transaction in a block, in order to possibly frontrun a pending transaction, thereby eliminating competition and rendering permutation ineffective, or by leveraging congestion to reduce the effectiveness of the random permutation since congesting the block with MEV extracting transactions increases the probability of successful extraction when facing permutation.

2.6 Obol / Nethermind: using DV clusters for encrypted transaction mempools

The Obol community proposal on “Using DV clusters for encrypted transaction mempools” [11], similarly to our proposal, attempts to use Distributed Validators to implement threshold encryption based MEV mitigation solutions. It proposes two main approaches:

1. Each builder DV cluster collects encrypted transactions whose bodies are hidden but gas limits are public. The leader orders transactions by expected profitability and selects a minimal candidate set, which the DV cooperatively decrypts using a threshold key generated via DKG to attempt block construction. If block assembly fails, the leader requests additional transactions to be decrypted, repeating this process until a valid block is built. The resulting block body is then encrypted for a set of trusted relay DVs and forwarded accordingly. Each relay DV selects the most profitable encrypted block, performs threshold decryption within its committee to verify correctness, and, upon success, exposes the block header to proposers for signing, allowing block candidates to be updated within the slot.
2. DVs as keypers. Each source encrypts transactions under the public key of a trusted DV cluster, ensuring that transaction contents remain hidden prior to ordering. The proposer constructs an encrypted block and commits to a specific transaction ordering by signing the encrypted block header. Once this commitment is made, the DV cluster cooperatively performs threshold decryption of the included transactions and returns the plaintexts (or posts them to a data-availability layer), enabling construction of the final cleartext block. Because the proposer’s commitment fixes the ordering before decryption, any deviation—such as omitting a committed transaction or re-ordering transactions—can be demonstrated by comparing the signed encrypted commitment with the decrypted outputs. Such misbehavior yields publicly verifiable evidence, which can be submitted to a restaking-based AVS that enforces slashing of the offending proposer or DV operators.

The proposal explicitly notes several limitations:

- Latency. Transactions must be known 2–3 slots in advance and block building involves multiple encryption/decryption and DA interactions, increasing slot-level latency.
- Reliance on a single proposer for block building. Even with DVs as keypers, the slot proposer remains responsible for assembling the final block being limited in his own resources.

- Partial coverage of public mempool. In practice, sources need to route their transactions to DVs they trust; the proposal does not fully address how plain public-mempool transactions are handled side-by-side with encrypted ones without leaking opportunities.

3 Solution: DVT-Based Encrypted Mempool Block Construction

We propose an encrypted-mempool block construction protocol in which transaction confidentiality and ordering are jointly enforced by a Byzantine-resilient proposer cluster based on Distributed Validator Technology (DVT). The protocol operates entirely on top of Ethereum’s existing consensus and execution model, requiring no changes to the EVM or transaction validity rules, and is therefore fully backward compatible.

The core objective of the design is to mitigate mempool-level and proposer-level MEV by removing unilateral information and ordering advantages. To this end, the protocol combines two complementary mechanisms. First, transaction contents are encrypted prior to mempool admission using recent advances in Batched Threshold Encryption (BTE) [12]. This prevents mempool observers and block builders from conditioning inclusion or ordering decisions on transaction semantics. Second, block construction is performed collectively by a DVT proposer cluster using a Byzantine-resilient multi-proposer protocol, eliminating the single-proposer ordering power that enables censorship and last-mover manipulation.

Users submit encrypted placeholder transactions to the public mempool. These placeholders are ordinary Ethereum transactions whose calldata contains an encrypted representation of a complete Ethereum transaction, encrypted under a published public key of an upcoming DVT proposer cluster that is eligible to propose a future block. Targeting an upcoming proposer ensures that only the intended cluster can perform decryption after ordering is fixed, while allowing transactions to propagate and compete in the public mempool as usual.

The DVT proposer cluster jointly constructs a block over these placeholders, fixing the transaction set and ordering while payloads remain confidential. Only after ordering is determined does the cluster perform threshold decryption and materialize the final block by deterministically replacing encrypted placeholders with their decrypted plaintext counterparts. Validators sign the block only after this replacement step is verified to be correct.

By separating ordering from execution visibility, and by distributing ordering authority across multiple proposers, the protocol removes the primary sources of information asymmetry exploited by frontrunning, sandwiching, and transaction reordering attacks, while preserving Ethereum’s native execution and fee semantics.

3.1 Encrypted Transactions and DV Cluster Block Construction

We now describe the protocol as a sequence of steps capturing the lifecycle of a transaction, from user-side encryption and mempool submission to block construction, post-commitment decryption, and block materialization by the DVT proposer cluster. Before detailing these steps, we briefly clarify the two core design components on which the protocol rests: an encrypted mempool that enforces inclusion-conditional confidentiality, and a multi-proposer block construction process that distributes ordering authority across the DVT cluster. Together, these mechanisms ensure that transaction contents remain hidden until ordering is determined, while preventing any single proposer from exerting unilateral control over transaction inclusion or execution order.

Encrypted mempool overview. To mitigate MEV arising from premature disclosure of transaction semantics, the protocol allows users to submit transactions whose execution-relevant contents are encrypted prior to mempool admission and carried as the payload of otherwise standard placeholder transactions. Rather than replacing the public mempool, the protocol operates entirely on top of it: encrypted transactions are propagated, validated, and ordered using existing Ethereum networking and fee

mechanisms. Confidentiality is achieved via batched threshold encryption (BTE), as instantiated in [2], which enables inclusion-conditional decryption with sublinear communication overhead. In this construction, users encrypt transactions independently under a shared public key, but decryption is performed jointly by the DVT proposer cluster only after a concrete batch of ciphertexts has been finalized during block construction. A single threshold decryption suffices to unlock all transactions in the batch, while ciphertexts not included in the finalized batch remain hidden, thereby preserving pending-transaction privacy.

Multi-proposer block construction overview Complementing the encrypted mempool, the protocol replaces Ethereum’s single-proposer block construction with a Byzantine-resilient multi-proposer process executed by a Distributed Validator Technology (DVT) cluster. Instead of granting a single proposer unilateral control over transaction inclusion and ordering, multiple proposers independently contribute candidate transaction sets, which are then combined through a commit-and-reveal protocol backed by Byzantine fault tolerant state machine replication. This process fixes the block’s transaction set and ordering before any encrypted payloads are decrypted, ensuring that no individual proposer can condition ordering decisions on transaction contents or exploit last-mover advantages. By distributing ordering authority across the cluster and binding proposers to their committed views, the construction removes unilateral proposer control over transaction inclusion and ordering, mitigating censorship, while remaining fully compatible with Ethereum’s existing consensus and execution semantics.

Step 0: User-side encryption and mempool submission. In order to submit a transaction, users encrypt the execution-relevant calldata of their transactions under a public encryption key shared by the DVT cluster. Crucially, encryption targets the public key of an upcoming DVT proposer cluster that is expected to propose a future block while running the protocol. This is possible on the Ethereum network since proposer identities rely on the consensus-layer randomness algorithm RANDAO which fixes validator selection two epochs in advance [17].

Concretely, as defined in [12], given a transaction payload m and a sampled transaction index i (explained below), the user computes

$$(c, \pi) \leftarrow \text{Enc}(\text{pk}, m, i),$$

where pk is the threshold public encryption key, c is the resulting ciphertext, and π is a non-interactive zero-knowledge proof attesting to correct encryption.

The sampled index i serves as the punctured index of the underlying puncturable, key-homomorphic pseudorandom function (PRF). It identifies the unique PRF evaluation point whose value is intentionally hidden prior to batch decryption. During encryption, the PRF key is punctured at index i , so that the PRF can be evaluated at all other indices but not at i itself. This property is essential for batched threshold decryption: it ensures that only ciphertexts explicitly included in the finalized batch can be decrypted, while ciphertexts that remain in the mempool retain confidentiality.

In addition, the user computes a commitment to the plaintext payload

$$h_m = H(m),$$

which is included in the transaction calldata.

The encrypted placeholder transaction is submitted as a standard Ethereum transaction whose calldata contains (c, π, h_m) . Metadata required for validity and fee payment—such as sender signature, nonce, gas limit, and gas price—remains in plaintext. The transaction is then signed by the user and broadcast to the public Ethereum mempool.

From the network’s perspective, encrypted placeholder transactions are standard Ethereum transactions. They are gossiped, validated, and admitted to the mempool by existing clients without modification.

Step 1: Local transaction collection. At the beginning of the slot, every operator in the DVT proposer cluster independently collects transactions from its local view of the public mempool. These may include encrypted transactions, as well as regular plaintext transactions. Each operator constructs a local inclusion list IL_i , representing the set of transactions it proposes for inclusion in the block.

Step 2: Commitment to proposed contents. To eliminate last-mover advantages—where a proposer conditions its inclusion decision on knowledge of other proposers’ choices—the protocol enforces a commitment phase during the first sub-interval of the consensus slot, where each proposer submits a state machine operation that records a cryptographic commitment to its local inclusion list. Concretely, proposer i computes a hash

$$h_i = H(\text{IL}_i)$$

and submits an operation that stores h_i in the i -th entry of the hashes array. These operations are totally ordered and replicated by a Byzantine fault-tolerant state machine replication protocol (e.g. [18], [19]). Once this sub-interval completes, each proposer is bound to either reveal the transaction payloads corresponding to its committed hash or reveal nothing.

Step 3: Reveal of inclusion lists. During the second sub-interval of the same consensus slot, each proposer may submit a state machine operation revealing its inclusion list IL_i . The state machine verifies that the revealed list hashes to the previously recorded commitment h_i and, if so, stores IL_i in the i -th entry of the inclusion list array. Proposers that fail to reveal a valid list leave their corresponding entry empty. By the end of this sub-interval, all correct and synchronous proposers have non-null entries in the inclusion list array.

Step 4: Deterministic merging. Given the agreed inclusion lists, the DVT cluster deterministically computes the final block contents. The transaction set is obtained by merging the inclusion lists according to a fixed, public rule, such as taking their union and prioritizing inclusion by transaction fees.

Step 5: Block commitment. The output of the construction process is a fully specified block body whose transaction set and ordering are fixed by agreement within the DVT cluster. At this point, encrypted transactions remain opaque, but their position in the block is final. This committed ciphertext block serves as the binding reference for subsequent threshold decryption: transaction contents are revealed only after ordering has been determined.

Step 6: Post-commitment batched threshold decryption. Once the block contents and transaction ordering are fixed, the DVT proposer cluster performs batched threshold decryption over the encrypted transactions included in the committed block, following the BTE construction of [12]. Let

$$\mathcal{B} = \{c_1, \dots, c_B\}$$

denote the batch of encrypted transactions selected for inclusion, where each ciphertext

$$c_j = (k_j^\star, \gamma_j, \text{ct}_j, \pi_j)$$

contains a punctured PRF key $k_j^\star$, a masked payload γ_j , a threshold ElGamal ciphertext ct_j encrypting the PRF key material, and a proof π_j of correct encryption. Exploiting the multiplicative homomorphism of threshold ElGamal, the cluster locally aggregates the ciphertexts to form a single ciphertext

$$\text{ct}_{\mathcal{B}} = \prod_{j=1}^B \text{ct}_j,$$

which encrypts the group element

$$K = g^{\sum_{j=1}^B k_j}.$$

Each operator ℓ , holding a secret key share sk_ℓ generated via distributed key generation, locally derives a single partial decryption

$$d_\ell = \text{DecShare}(\text{sk}_\ell, \text{ct}_\mathcal{B}),$$

which is generated only after block commitment, scoped to the committed batch, and independent of the batch size B . Given a quorum S of at least t valid partial decryptions $\{d_\ell\}_{\ell \in S}$, the aggregate group element K is recovered via Lagrange interpolation. Using K , the plaintext payload corresponding to each ciphertext $c_i \in \mathcal{B}$ is reconstructed as

$$m_i = \gamma_i + \sum_{j \neq i} \text{PEval}(k_j^*, i) - \text{ExpEval}(K, i),$$

which follows from the key-homomorphic and puncturing properties of the PRF. Ciphertexts not included in $\mathcal{B}$ remain computationally hidden due to their unresolved punctured indices. No operator can decrypt transactions independently, and no decryption oracle is exposed prior to commitment, ensuring pending-transaction privacy with $O(n)$ total decryption communication.

Step 7: Plaintext reconstruction and block materialization. After the plaintext payloads corresponding to encrypted placeholder transactions in the committed block are reconstructed off-chain, the DVT proposer cluster materializes the final block by deterministically processing the ordered transaction list fixed in Step 5. Each reconstructed payload is required to represent a complete Ethereum transaction. For every encrypted placeholder transaction, the cluster applies the following deterministic replacement procedure:

Algorithm 1 Placeholder Transaction Resolution

```

1: if decryption succeeds and  $H(m) = h_m$  and  $m$  is syntactically valid then
2:   if gas limit matches placeholder gas limit then
3:     Replace placeholder transaction with  $m$ 
4:   else
5:     Select the most profitable transaction such that
6:     (gas limit  $\times$  gas price) is payable by the user
7:   end if
8: else
9:   Retain the placeholder transaction unchanged in the block
10: end if

```

This replacement rule is deterministic, ensuring that no proposer discretion is introduced during materialization.

Step 8: Distributed block signing and publication. After block materialization is complete, the DVT proposer cluster collectively authorizes the finalized block according to the underlying DVT signing mechanism. Each operator locally verifies that the block body is consistent with the committed ordering from Step 5 and the deterministic replacement rules applied in Step 7. Upon successful verification, each operator produces a partial block signature using its local validator key share.

Partial signatures from a quorum of at least t operators are then aggregated into a single valid proposer signature using an existing threshold signature scheme (e.g., threshold BLS signatures). The resulting block is broadcast to the Ethereum network as a standard block proposal. From the perspective of Ethereum consensus and execution clients, the block is indistinguishable from one produced by a single proposer and is validated and executed according to existing protocol rules.

3.2 Challenges and Practical Considerations

While the proposed DVT-based encrypted mempool protocol achieves strong cryptographic and systems-level guarantees against mempool and proposer-level MEV, several practical challenges must be addressed

for deployment.

Economic competitiveness of encrypted blocks. A primary concern is whether blocks constructed under the proposed protocol remain economically competitive with blocks produced in today’s MEV-driven ecosystem. By design, encrypted transactions prevent proposers and builders from conditioning inclusion and ordering decisions on transaction semantics, thereby eliminating many profitable MEV extraction strategies. While this improves fairness and user protection, it may reduce the short-term revenue available to block producers compared to existing PBS pipelines. Sustained adoption therefore depends on whether encrypted blocks can attract sufficient transaction fees, whether users are willing to pay a premium for privacy and MEV resistance, and whether protocol-level incentives or social coordination mechanisms emerge to favor fair ordering over extractive behavior.

Deployment of novel cryptographic primitives. The protocol relies on batched threshold encryption (BTE), a recently proposed cryptographic primitive that departs from widely deployed threshold encryption schemes. Although BTE achieves optimal communication complexity and strong pending-transaction privacy, it introduces additional implementation complexity that remains to be explored.

Consensus slot timing and performance constraints. Finally, the protocol must operate within the strict timing constraints of Ethereum’s consensus slots. Multi-proposer block construction introduces additional coordination rounds, and post-commitment batched decryption requires cryptographic computation whose cost grows with the batch size. Although BTE is explicitly designed to minimize communication overhead during the latency-critical decryption phase, plaintext reconstruction incurs local batch-level computation, which BEAT-MEV’s experimental evaluation shows can be completed within practical time bounds for block-sized batches. Ensuring that block construction, decryption, materialization, and distributed signing can reliably complete within a single slot—under realistic network delays and validator heterogeneity—will be a key systems challenge.

4 Evaluation

This section describes how the proposed protocol will be evaluated around four research questions (RQs). The evaluation is designed to assess (i) whether the protocol can operate within Ethereum’s slot-level timing constraints, (ii) the overhead introduced by multi-proposer coordination and cryptographic mechanisms, (iii) the protocol’s robustness under realistic faults and adversarial conditions, and (iv) the economic implications of the protocol on users and proposers.

4.1 Evaluation Setup and Prototype

The evaluation will be conducted using a prototype implementation composed of four main components: (i) a multi-proposer block building layer integrated into a distributed validator technology (DVT) cluster, (ii) a batched threshold encryption (BTE) module based on [12] and the available proof-of-concept¹, (iii) a user-side encryption client targeting an upcoming proposer/DVT cluster, and (iv) an experiment harness for orchestration, measurement, and fault injection.

Experiments will be conducted in two environments: (i) a local distributed setup to isolate protocol and cryptographic overheads, and (ii) a geo-distributed deployment across multiple AWS availability zones to capture realistic network latency and jitter.

4.2 RQ1: Is the protocol practical within Ethereum’s slot-level timing constraints?

The first research question evaluates whether the proposed protocol can complete all latency-critical operations within Ethereum’s consensus slot deadlines.

¹<https://zenodo.org/records/14672008>

Multi-proposer block construction latency. We will integrate the multi-proposer block building mechanism into an open-source DVT stack, using `ssv`² as the base implementation. The prototype will add a coordination layer among DVT operators that produces a single deterministic block proposal per slot, which is then signed using the existing DVT threshold BLS workflow.

We will measure the end-to-end latency from slot start to finalized block output and signature, and evaluate whether block construction completes reliably within a single slot under varying network conditions.

Post-commitment decryption latency. To assess the feasibility of batched threshold decryption within Ethereum’s timing constraints, we will integrate the BTE module into the block building pipeline. The evaluation will measure the time required to generate decryption shares, aggregate them, and reconstruct plaintext transactions after block commitment.

Metrics. For this RQ, we will measure: (i) end-to-end slot latency, (ii) commit-to-plaintext latency, (iii) success rate of producing a valid block within a slot, and (iv) latency distributions (median, p95, p99) under both local and geo-distributed deployments.

4.3 RQ2: What overhead does multi-proposer coordination and cryptography introduce?

The second research question focuses on quantifying the computational, communication, and client-side overhead introduced by the protocol.

Coordination and resource overhead. We will measure bandwidth usage, CPU time, and peak memory consumption per DVT operator during multi-proposer coordination. This includes the overhead of proposal exchange, deterministic block construction, and threshold signing.

Cryptographic overhead of BTE. Using the standalone BTE module derived from the proof-of-concept in [12], we will benchmark: (i) client-side encryption latency and ciphertext size, (ii) per-operator decryption-share generation latency, (iii) reconstruction latency, including aggregation and combine steps, and (iv) bandwidth overhead for decryption artifacts.

User-side overhead. We will implement an encryption client that targets an upcoming proposer/DVT cluster. The evaluation will measure: (i) time to obtain and validate public encryption material, (ii) end-to-end client overhead (encryption plus submission), and (iii) added latency relative to plaintext transaction submission.

Parameter sweeps and stress testing. All experiments above will be repeated under multiple configurations:

- **DVT cluster size:** varying the number of operators and threshold parameters.
- **Batch size:** varying the number of encrypted transactions per block to study scalability.
- **Optimizations:** evaluating the impact of BTE optimizations described by Bormet et al. [12] and comparing observed results against those reported in prior work.

For each configuration, we will report latency, throughput, and resource consumption, and identify regimes in which overhead approaches practical limits.

²<https://github.com/ssvlabs/ssv>

4.4 RQ3: How robust is the protocol under faults and adversarial behavior?

The third research question evaluates the protocol’s robustness using controlled fault injection and adversarial workloads.

Censorship and omission attempts. We will simulate scenarios in which a subset of DVT operators attempts to censor or deprioritize specific ciphertexts. The evaluation will assess whether multi-proposer coordination and deterministic inclusion rules mitigate unilateral control over transaction inclusion.

Decryption-share failures. We will evaluate scenarios in which operators withhold decryption shares or release malformed shares. The evaluation will measure liveness, time-to-decrypt, and recovery behavior as the number of faulty operators increases, under the assumed threshold.

Malformed and inconsistent inputs. We will inject malformed ciphertexts (e.g., invalid proofs or encodings), as well addition, we will test attempts by malicious block-building nodes to include plaintext transactions that do not correspond to the committed ciphertext set. We will assess whether such inputs are reliably detected and rejected during verification and block assembly.

Metrics. For each adversarial scenario, we will report: (i) performance impact (latency, throughput, success rate), (ii) correctness outcomes (invalid blocks rejected, premature disclosure prevented), and (iii) safety and liveness guarantees maintained within the modeled fault bounds.

4.5 RQ4: What is the economic cost of the protocol for users and validators?

The fourth research question evaluates the economic implications of the proposed protocol, focusing on the additional costs incurred by users and validators relative to standard Ethereum transaction submission.

User cost. We will quantify the incremental cost to users arising from encrypted transaction submission. This includes: (i) increased calldata size due to ciphertexts and accompanying cryptographic material, (ii) any additional gas costs incurred by verification or handling of encrypted transactions, and (iii) indirect costs resulting from latency-induced fee dynamics (e.g., delayed inclusion under congestion).

Using the implemented encryption client, we will measure the gas usage of encrypted transactions and compare it against equivalent plaintext transactions under identical network conditions. Costs will be reported in gas units and converted to ETH using contemporaneous base fee and priority fee data.

Validator and proposer cost. We will measure the additional computational and bandwidth costs borne by DVT operators and proposers due to multi-proposer coordination and batched threshold decryption. These costs will be translated into estimated operational expenses (e.g., CPU time and bandwidth consumption per slot) to assess whether they are likely to be amortized by existing proposer rewards or require explicit incentivization.

Cost scaling under load. To understand how costs evolve under realistic conditions, we will evaluate user and validator costs as a function of: (i) transaction batch size, (ii) mempool congestion levels derived from historical throughput traces, and (iii) DVT cluster size and threshold parameters.

Metrics. For this RQ, we will report: (i) additional gas cost per transaction relative to plaintext submission, (ii) total cost per block attributable to encrypted transaction handling, (iii) distribution of user costs under varying congestion levels (median, p95, p99), and (iv) validator-side overhead translated into estimated per-slot operational cost.

5 Work Schedule

The work will proceed in staged phases, beginning with protocol refinement and an in-depth study of existing codebases, including the SSV DVT implementation ³ and the batched threshold encryption prototype presented in [12]. This will be followed by the development of a full prototype integrating multi-proposer block construction with encrypted transaction handling. The next step will be performing a systematic experimental evaluation of the prototype and collecting performance metrics under realistic network and fault conditions. The final stage will be consolidating the results and completing a finalized beta version of the dissertation. A Gantt chart is shown in Figure 1.

6 Conclusion

This proposal outlines a backward-compatible approach to mitigating MEV by jointly addressing transaction confidentiality and ordering power. By combining an encrypted mempool based on batched threshold encryption with a Byzantine-resilient, multi-proposer block construction implemented through a DVT cluster, the design removes the proposer’s unilateral ability to condition ordering decisions on transaction contents while avoiding changes to Ethereum’s consensus or execution layers. Addressing privacy through encryption and fairness through distributed block construction allows the protocol to mitigate MEV strategies and censorship without relying solely on economic incentives or trusted intermediaries.

Beyond its security benefits, the proposed architecture is intentionally grounded in deployability. It builds on recent cryptographic advances with strong security foundations and demonstrated practical viability, and combines them with Byzantine-resilient multi-proposer block construction deployed via existing DVT frameworks as a realistic path toward incremental adoption. The proposed evaluation will validate not only the protocol’s cryptographic and systems performance, but also its economic implications for users and operators. Taken together, this work aims to translate a protocol-level approach to MEV mitigation that combines confidentiality and distributed block construction into a concrete and deployable component of the Ethereum ecosystem, contributing to a fairer and more robust transaction pipeline.

³<https://github.com/ssvlabs/ssv>

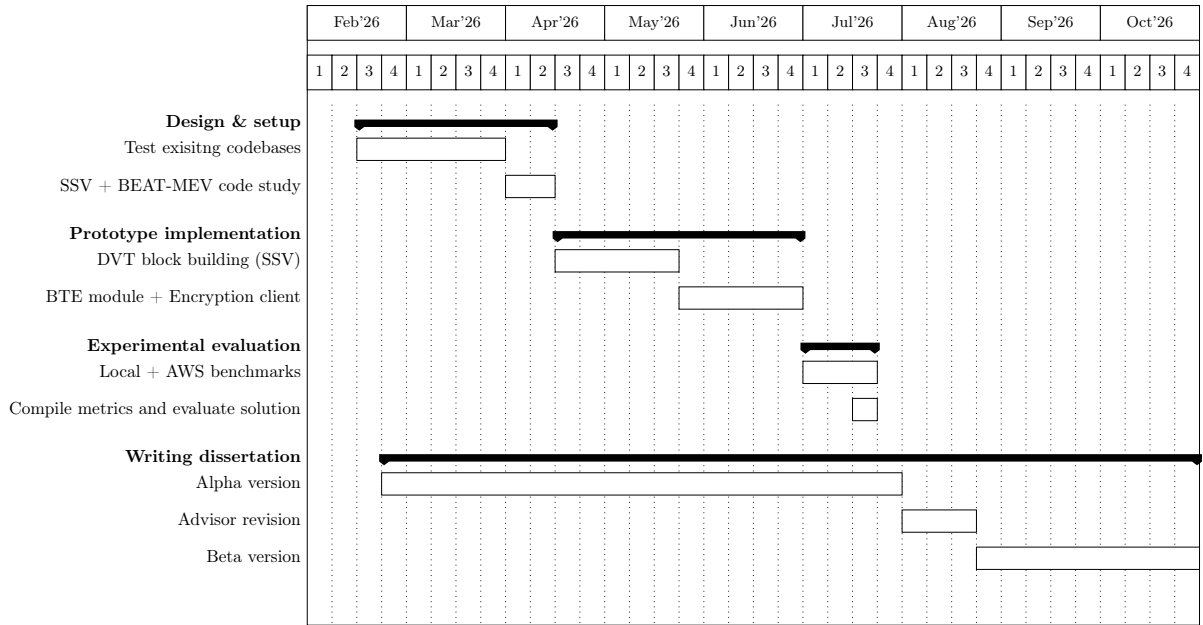


Figure 1: Planned schedule

Bibliography

- [1] defillama. (2026) defillama. Accessed: 2026-01-07. [Online]. Available: <https://defillama.com/>
- [2] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels, “Flash boys 2.0: Frontrunning, transaction reordering, and consensus instability in decentralized exchanges,” 2019. [Online]. Available: <https://arxiv.org/abs/1904.05234>
- [3] S. Yang, F. Zhang, K. Huang, X. Chen, Y. Yang, and F. Zhu, “SoK: MEV countermeasures,” in *Proceedings of the Workshop on Decentralized Finance and Security (DeFi ’24)*. New York, NY, USA: ACM, 2024, pp. 1–10. [Online]. Available: <https://doi.org/10.1145/3689931.3694911>
- [4] Flashbots, “Mev-boost: Proposer-builder separation for ethereum,” <https://docs.flashbots.net/flashbots-mev-boost/introduction>, 2022.
- [5] S. Yang, K. Nayak, and F. Zhang, “Decentralization of ethereum’s builder market,” *arXiv preprint arXiv:2405.01329*, 2025.
- [6] R. Miller, “Post mortem: April 3rd, 2023 mev-boost relay incident and related timing issue,” <https://collective.flashbots.net/t/post-mortem-april-3rd-2023-mev-boost-relay-incident-and-related-timing-issue/1540>, 2023, accessed: 2026-01-07.
- [7] F. Garcia, E. Kogias, C. Kokoris-Kogias, and Others, “BlindPerm: Efficient MEV Mitigation with an Encrypted Mempool and Permutation,” 2023.
- [8] H. Zhang, L. Merino, Z. Qu, M. Bastankhah, V. Estrada-Galiñanes, and B. Ford, “Flash Freezing Flash Boys (F3B),” 2022.
- [9] J. Yu, S. Luo, K. Chen, H. Lin, W. Liang, and M. Li, “FairBlock: Bandwidth-Efficient Threshold Decryption for Blockchains,” 2022.
- [10] J. Krämer, S. Barz, H. Diedrich, and S. P. Team, “Shutter Network: Private Transactions from Threshold Cryptography,” 2024.
- [11] Obol Collective, “Using DV Clusters for Encrypted Transaction Mempools,” <https://community.obol.org/t/proposal-using-dv-clusters-for-encrypted-transaction-mempools/108>, 2023.
- [12] J. Bormet, S. Faust, H. Othman, and Z. Qu, “BEAT-MEV: Epochless approach to batched threshold encryption for MEV prevention,” *Cryptology ePrint Archive*, Paper 2024/1533, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1533>
- [13] A. R. Choudhuri, S. Garg, J. Piet, and G.-V. Policharla, “Mempool privacy via batched threshold encryption: Attacks and defenses,” in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 3513–3529. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/choudhuri>
- [14] V. Buterin, “Proposer/builder separation,” <https://ethresear.ch/t/proposer-block-builder-separation-friendly-fee-market-designs/>, 2021.
- [15] P. Janicot and A. Vinyas, “Private mev protection rpcs: Benchmark study,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.19708>
- [16] F. Lührs, L. Bezenberger, F. Mosterts, S. Faust, and A. Erwig, “The Road Towards an Encrypted Mempool on Ethereum,” https://docs.shutter.network/docs/shutter/research/the_road_towards_an_encrypted_mempool_on_ethereum, 2023, shutter Documentation.
- [17] Ethereum Foundation. (2024) Ethereum foundation documentation. Accessed: 2026-01-07. [Online]. Available: <https://ethereum.org/developers/docs/consensus-mechanisms/pos/block-proposal/>

- [18] D. S. Antunes, A. N. Oliveira, A. Breda, M. G. Franco, H. Moniz, and R. Rodrigues, “Alea-bft: Practical asynchronous byzantine fault tolerance,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.14538>
- [19] M. Castro, B. Liskov *et al.*, “Practical byzantine fault tolerance,” in *OsDI*, vol. 99, no. 1999, 1999, pp. 173–186.